Question List

1. Description of Bit and Byte ordering

2. Reasons to organise the AES state differently

3. AES implementation of

    a. Addition and Multiplication
    b. xTimes implementation

4. Implementations

    a. MixColumns implementation
    b. Finite field Inversion implementation, in “ GF28_inv(x) ”
    c. Key Expansion implementation
    d. All AES round implementation, in “ AES_encr(AES_state, key) “
    e. Verify paper exemples

5. S-Box generation function implementation

6. Combining SubBytes and ShiftRows implementation

7. Considerations about XOR with an m

    a. consider what happens if we apply AddRoundKey, SubBytes, ShiftRows to “ a $\oplus$ m “ rather than “a”
    b. what happens if we apply MixColumns to : (a0 $\oplus$m0, a1 $\oplus$m1, a2 $\oplus$m2, a3 $\oplus$m3)

1. How the bits in an input block are organised? How do the byte ordering and the ordering of the bits within each byte differ?

The input bit sequence is a bit-string that has the first bit of the sequence as the first element of the bit-string.

Instead, the bits within a byte are arranged in another way, we have that the first element is also the most significant bit.

This is because here the memory management is done in a *Big Endian* way.

Arrays of bytes will be represented in the following form:

$$a_0 a_1 a_2 ... a_{15}$$

The bytes and the bit ordering within bytes are derived from the 128-bit input sequence

$$input_0\ input_1\ input_2\ ...\ input_{126}\ input_{127}$$

Input bit sequence	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	...
Byte number	0								1								2								...
Bit numbers in byte	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	...

2. Could there be (good) reasons to organise the AES state differently within the memory (of a computer)?

Yes, there may be.

For example, we may try to improve our security by reordering the state in memory.

This way one could try to achieve resilience against side channel attacks that aim at obtaining leakages from power or time measurements.

We may find a way to make it difficult to identify the correlation between data and side-channel leaks.

3. Check out the implementation of the finite field arithmetic functions in AES.

a. Explain addition and multiplication

Addiction :

The addiction over integer numbers is here substituted by the field addiction.

In AES it is usual to interpret every byte of the state array as an element of the **Galois Field GF(2^8)**.

Here we need to obtain a polynomial representation of the byte.

Since it is composed of 8 bits, it is possible to understand each bit as an element of the **Galois Field GF(2)**, which is the finite field with only 2 elements.

Thus, by replicating GF(2) for 8 times we obtain a vector which in its entirety can be seen as an element of GF(2^8).

Our byte {bit_7 bit_6 bit_5 bit_4 bit_3 bit_2 bit_1 bit_0} can be interpreted as a *polynomial*:

$$b(x) = b7\ x^7 + b6\ x^6 + b5\ x^5 + b4\ x^4 + b3\ x^3 + b2\ x^2 + b1\ x + b0$$

From what has been said, it follows that to perform the sum of two finite field elements belonging to GF(2^8) is necessary to go to make a polynomial addition, which corresponds to the XOR of the two elements.

So every bit of the first finite field element will go into XOR with the corresponding bit of the second finite field element, thus obtaining a byte:

$$\{a7 \oplus b7\ a6 \oplus b6\ a5 \oplus b5\ a4 \oplus b4\ a3 \oplus b3\ a2 \oplus b2\ a1 \oplus b1\ a0 \oplus b0\}$$

Multiplication :

As with addition, also the multiplication is replaced by the finite field arithmetic operation, the field multiplication.

When we multiply 2 finite field elements we do a termwise multiplication and then make a modular reduction of the result by the ***irreducible polynomial***:

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

So if I have 2 polynomials (each one represents a byte) named **e(x)** and **d(x)**, I could calculate the reduction of their product as polynomials like this:

$$e(x)d(x) \bmod (m(x))$$

The modular reduction is necessary because we want to work with the finite field consisting of 256 elements.

b. Consider xTimes, find out

i. what this is using the AES standard

xTimes (or also called FMUL-X) is a component that receives in input a finite field element (a polynomial) and multiplies it by **X**.

It produces the polynomial representation of the output byte.

I would go deeper the implementation in the next question.

ii. provide an implementation that does not use the provided GF28 multiply() function.

The provided xTimes implementation goes to:

- **Shift** to the **left** of the input polynomial, this consists in going to multiply by x the polynomial, thus raising the degree of each x in the **polynomial**.

  We store this value in a temporary temp variable.

- Get the most significant bit of the input polynomial

- If the **msb** is = 1, need to perform the **reduction** by the **irreducible polynomial**.

  Else if is 0, we know that doing something XOR a bit-sting of zeroes, we obtain the starting bitstring.

- Make the reduction by doing " temp XOR x "

In this way, a reduction is always performed. We could summarize in this way:

- If msb = 0 : temp XOR 0 = temp
- If msb = 1 : temp XOR Irreducible polynomial (effective reduction step)

4. Implement MixColumns

This transformation uses finite field multiplication many times in such a way that the column vector is obtained by producing the product of a column vector for a matrix.

The column vector is here a column taken from the state matrix.

We can get the output of the transformation applied to the individual bytes in this way:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad \text{for } 0 \le c < 4,$$

so that the individual output bytes are defined as follows:

$$\begin{aligned} s'_{0,c} &= (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c} \\ s'_{1,c} &= s_{0,c} \oplus (\{02\} \bullet s_{1,c}) \oplus (\{03\} \bullet s_{2,c}) \oplus s_{3,c} \\ s'_{2,c} &= s_{0,c} \oplus s_{1,c} \oplus (\{02\} \bullet s_{2,c}) \oplus (\{03\} \bullet s_{3,c}) \\ s'_{3,c} &= (\{03\} \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \bullet s_{3,c}). \end{aligned}$$

4.b GF28_inv(x) implementation (finite field inversion)

The finite field inversion can be performed using the polynomial version of the Extended Euclidean Algorithm.

But, in my implementation, I preferred to use a **Fermat-style approach** (based on Fermat's Little Theorem).

The inversion on GF(256) is defined on elements that differ from 0, therefore on a set composed of 255 elements.

Inversion can be expressed by an exponentiation.

For a byte $b \neq \{00\}$, its multiplicative inverse is the unique byte, denoted by b^{-1}, such that

$$b \bullet b^{-1} = \{01\}. \tag{4.10}$$

The Multiplicative inverse in GF(2^8) can be expressed as :

$$b^{-1} = b^{254}.$$

4.c Key expansion and Key scheduling implementation

Key Expansion is a routine applied to the key to generate keys for each round.

A word is composed by 4 bytes, in each round we would use 4 words to compose the key state matrix.

We use 2 word transformations, **Word Rotation** and **Word Substitution**.

KeyExpansion() has been implemented inside the code following the FIPS 197 standard.

Algorithm 2 Pseudocode for KEYEXPANSION()

```
1:  procedure KEYEXPANSION(key)
2:      i ← 0
3:      while i ≤ Nk − 1 do
4:          w[i] ← key[4 ∗ i..4 ∗ i + 3]
5:          i ← i + 1
6:      end while                          ▷ When the loop concludes, i = Nk.
7:      while i ≤ 4 ∗ Nr + 3 do
8:          temp ← w[i − 1]
9:          if i mod Nk = 0 then
10:             temp ← SUBWORD(ROTWORD(temp)) ⊕ Rcon[i/Nk]
11:         else if Nk > 6 and i mod Nk = 4 then
12:             temp ← SUBWORD(temp)
13:         end if
14:         w[i] ← w[i − Nk] ⊕ temp
15:         i ← i + 1
16:     end while
17:     return w
18: end procedure
```

4.d Implement all AES rounds

The code to implement all rounds is contained inside the function: **AES_encr(state, key)**

4.e Verify with paper examples

To test the correctness of the written code, I used the Cipher Example provided in the official paper of AES.

To obtain this result, you need to execute in my code, the function: **AES_encr(state, key)**

Round Number	Start of Round	After SubBytes	After ShiftRows	After MixColumns	Round Key Value
input	32 88 31 e0 43 5a 31 37 f6 30 98 07 a8 8d a2 34				2b 28 ab 09 7e ae f7 cf 15 d2 15 4f 16 a6 88 3c
1	19 a0 9a e9 3d f4 c6 f8 e3 e2 8d 48 be 2b 2a 08	d4 e0 b8 1e 27 bf b4 41 11 98 5d 52 ae f1 e5 30	d4 e0 b8 1e bf b4 41 27 5d 52 11 98 30 ae f1 e5	04 e0 48 28 66 cb f8 06 81 19 d3 26 e5 9a 7a 4c	a0 88 23 2a fa 54 a3 6c fe 2c 39 76 17 b1 39 05

```
AES State beginning
[['32', '88', '31', 'E0'],
 ['43', '5A', '31', '37'],
 ['F6', '30', '98', '07'],
 ['A8', '8D', 'A2', '34']]
AES State after AddRoundKey
[['19', 'A0', '9A', 'E9'],
 ['3D', 'F4', 'C6', 'F8'],
 ['E3', 'E2', '8D', '48'],
 ['BE', '2B', '2A', '08']]

----------- Start of round 1 -----------

AES State after SubBytes : Round 1
[['D4', 'E0', 'B8', '1E'],
 ['27', 'BF', 'B4', '41'],
 ['11', '98', '5D', '52'],
 ['AE', 'F1', 'E5', '30']]
AES State after ShiftRows : Round 1
[['D4', 'E0', 'B8', '1E'],
 ['BF', 'B4', '41', '27'],
 ['5D', '52', '11', '98'],
 ['30', 'AE', 'F1', 'E5']]
AES State after MixColumns : Round 1
[['04', 'E0', '48', '28'],
 ['66', 'CB', 'F8', '06'],
 ['81', '19', 'D3', '26'],
 ['E5', '9A', '7A', '4C']]
```

5. Implement a new function that generates the sbox table

The **Substitution Box** is defined as the composition of two functions: S-Box(a) = f (g (a))

First of all, it is necessary to compute g(a), that is a finite field inversion.

We use for this purpose the function " **GF28_inv(x)** " (previously defined).

Remember that it is defined only over not 0 elements, to be able to handle this I went to define the function **gf_inverse(num)**.

Then we go to apply the affine transformation.

It receives the bitstring produced by g and performs the finite field multiplication with a matrix defined in such a way that only some elements can be activated in each row.

The resulting column vector will go into XOR with another column vector.

The code is inside the functions " **aff_transf(x)** " and " **gen_s_box(x)** "

$$\text{S-Box}(a) = f(g(a))$$

where

$$g(a) = 1 \oslash_{\mathbb{F}_{2^8}} a,$$

i.e., g is a field inversion, and

$$f\left(\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix}\right) = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \otimes_{\mathbb{F}_2} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix} \oplus_{\mathbb{F}_2} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

We did so in order to create diffusion across the finite field inversion output, this is intended to destroy the extraction of the algebra that the finite field inversion could introduce into the overall AES structure.

6. Consider combining SubBytes, ShiftRows

The idea is that instead of transforming the whole state with **SubBytes** and then with **ShiftRows**, I can go to apply the substitution (SubBytes) and shift (ShiftRows) simultaneously line by line.

The idea is that I replace an element and then apply the circular displacement.

To do so, I created the function " **sb_sr(state)** ", and then use it inside " **AES_encr_combined_sb_and_sr(AES_state, key)** "

7. I want you to consider what happens, for each round function, if we wanted to apply it to inputs where each byte is exclusive-ored with another value m.

    a. So consider what happens if we apply AddRoundKey, SubBytes, ShiftRows to " a ⊕ m " rather than "a".

We want to evaluate how the order of application of the XOR operations can affect the AES algorithm.

Specifically we know that these 3 transformations : "**AddRoundKey**, **SubBytes**, **ShiftRows**" are all **byte oriented in AES**. This means that if requested, it is possible to provide an implementation of AES which performs all 3 these byte oriented transformations at once for each single byte.

In contrast, the only one **not** to be **byte oriented** is **MixColumns**.

To answer the question, I chose to use an m matrix to mask the state matrix.

I arbitrarily chose to fill the matrix with the hexadecimal value 0x42 (66 in decimal).

In the code, I defined the function " **xor_with_m(state)** " that takes as input the state and returns the matrix obtained by making the XOR of each element in the state matrix with the respective in the matrix m (this last one composed by a constant equal to 0x42).

I want to now check if there is a difference between the outputs using :

- Input 1 : **a ⊕ m**
- Input 2 : **m**

- ***AddRoundKey***, we observe : AddRoundKey(a XOR m) == AddRoundKey(a) XOR m

  This is because the AddRoundKey is a linear transformation.

```
AES State after AddRoundKey(a XOR m) :  -    Way 1
[['5B', 'E2', 'D8', 'AB'],
 ['7F', 'B6', '84', 'BA'],
 ['A1', 'A0', 'CF', '0A'],
 ['FC', '69', '68', '4A']]
AES State after AddRoundKey(a) XOR m :  -    Way 2
[['5B', 'E2', 'D8', 'AB'],
 ['7F', 'B6', '84', 'BA'],
 ['A1', 'A0', 'CF', '0A'],
 ['FC', '69', '68', '4A']]
```

- ***SubBytes***, we observe : SubBytes(a XOR m) != SubBytes(a) XOR m
  This is because the SubBytes is a non-linear transformation.

  The state matrix " a " contains elements that are replaced with their replacement.
  If I then do the XOR of the value that I got with the corresponding in m, I will get a certain value.

  It will be different from the counterpart I would get by doing the substitution after having already done the : a $\oplus$ m.

```
AES State after SubBytes(a XOR m) : Round 1 -    Way 1
[['D4', 'E0', 'B8', '1E'],
 ['27', 'BF', 'B4', '41'],
 ['11', '98', '5D', '52'],
 ['AE', 'F1', 'E5', '30']]
AES State after SubBytes(a) XOR m : Round 1 -    Way 2
[['7B', 'DA', '23', '20'],
 ['90', '0C', '1D', 'B6'],
 ['70', 'A2', 'C8', '25'],
 ['F2', 'BB', '07', '94']]
```

- ***ShiftRows***, we observe : ShiftRows (a XOR m) == ShiftRows (a) XOR m

  This is because ShiftRows is a linear transformation.

```
AES State after ShiftRows(a XOR m) : Round 1 -    Way 1
[['96', 'A2', 'FA', '5C'],
 ['FD', 'F6', '03', '65'],
 ['1F', '10', '53', 'DA'],
 ['72', 'EC', 'B3', 'A7']]
AES State after ShiftRows(a) XOR m : Round 1 -    Way 2
[['96', 'A2', 'FA', '5C'],
 ['FD', 'F6', '03', '65'],
 ['1F', '10', '53', 'DA'],
 ['72', 'EC', 'B3', 'A7']]
```

- ***SubBytes & ShiftRows***, we observe : sb_sr (a XOR m) != sb_sr (a) XOR m

  This is because the SubBytes is a non-linear transformation.

```
AES State after SubBytes_&_ShiftRows(a XOR m) : Round 1 -    Way 1
[['D4', 'E0', 'B8', '1E'],
 ['BF', 'B4', '41', '27'],
 ['5D', '52', '11', '98'],
 ['30', 'AE', 'F1', 'E5']]
AES State after SubBytes_&_ShiftRows(a) XOR m : Round 1 -    Way 2
[['7B', 'DA', '23', '20'],
 ['0C', '1D', 'B6', '90'],
 ['C8', '25', '70', 'A2'],
 ['94', 'F2', 'BB', '07']]
```

b. And what happens if we apply Mix Columns to :
(a0 ⊕m0, a1 ⊕m1, a2 ⊕m2, a3 ⊕m3).

To perform the XOR by adding each column of the state matrix with a column of the m matrix we use the function " **columnwise_xor_with_m(state)** ".

We also try to compare the outputs in the context of the two different Inputs 1 and Inputs 2, we get that:

MixColumns (a XOR m) == MixColumns (a) XOR m

This is because **MixColumns** is a **linear transformation**.

```
AES State after MixColumns(a XOR m) : Round 1 -    Way 1
[['47', 'A0', '09', '6E'],
 ['25', '8B', 'B9', '40'],
 ['C2', '59', '92', '60'],
 ['A6', 'DA', '3B', '0A']]
AES State after MixColumns(a) XOR m : Round 1 -    Way 2
[['47', 'A0', '09', '6E'],
 ['25', '8B', 'B9', '40'],
 ['C2', '59', '92', '60'],
 ['A6', 'DA', '3B', '0A']]
```

Conclusions

Although it seems counterintuitive, it is not always useful to perform operations such as the XOR of a masking matrix with the state matrix.

The presence of non-linear transformations plays a crucial role in this.